MSCS-532

Algorithms and Data Structures

Spring 2026

Name: Victor Hugo Abad Navarrete

Assignment 4: Heap Data Structures Implementation Analysis and Applications

1. Theoretical Analysis

This section provides a theoretical analysis of the heapsort algorithm. The algorithm works in two phases. First, a max-heap is built from the original array. Second, the first element of the heap is always taken as the maximum value of the heap, so it is extracted from the heap n – 1 times, and after every extraction, MAX-HEAPIFY is called to maintain the heap property of the remaining elements. Hence, the total cost of the heapsort algorithm depends on the BUILD-MAX-HEAP and (n-1) MAX-HEAPIFY costs.

$$T(n) = T_{build}(n) + (n-1)T_{heapify}(n)$$

The build-max-heap algorithm calls happify for all the nodes starting in n/2 and ending in 1.First the cost of heapify is calculated. Heapify restores the heapify property of a node and its leaves by comparing the root and the leaves. If they don't satisfy the heap property, two nodes are interchanged, and heapify is called on the exchanged node. Since recursion only happens for one of the branches and the maximum number of elements in each of the branches' subtrees is 2n/3 (Cormen, Leiserson, & Rivest, 2022):

$$T_{hapify}(n) \leq T\left(\frac{2n}{3}\right) + c_1$$

By the substitution method with T(n) = O(lg n). Then the inequality T(n) <= a lg(n) follows.

$$T_{hapify}(n) \leq a\ lg\left(\frac{2n}{3}\right) + c_1$$

$$T_{hapify}(n) \leq a\lg(2) + a\lg(n) - a\lg(3) + c_1$$

$$T_{hapify}(n) \leq a\lg(n) + c_1$$

$$T_{hapify}(n) = O\ (\lg n)$$

For the build-max-heap component, the total cost is calculated by the total number of nodes at any height h, and each of those nodes can move down to the ends of the root (h times). The total number of levels is lg (n). Hence, the sum of total work is given by:

$$T_{build}(n) = \sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{n^{h+1}} \right\rceil ch$$

(Cormen, Leiserson, & Rivest, 2022)

An upper bound is given by:

$$T_{build}(n) \leq \sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{n^{h}} \right\rceil ch$$

$$T_{build}(n) \leq cn \sum_{h=0}^{\infty} \frac{h}{2^h}$$

$$T_{build}(n) \leq cn \frac{\frac{1}{2}}{\left(1-\frac{1}{2}\right)^2}$$

$$T_{build}(n) = O(n)$$

The total running time of heapsort is then:

$$T(n) = O(n) + (n-1)\ O\ (\lg(n))$$

$$T(n) = O(n \lg n)$$

In the worst case, initial data, build-heapify will always result in a heap array, and the number of elements to run heapify is constant regardless of the array. The cost of heapify remains the same and is bounded by O(n). Then, the number of times that the first element is extracted and the

heapify is run is fixed. This algorithm invariance means that regardless of how the data is presented, the worst, best, and average cases are bounded by O(n).

Space complexity and additional overheads

In heap sort, the array is sorted in place, and for every heapify iteration, the heap property is restored by just swapping elements. No new memory allocation is required besides the required memory for function calls. There are three sources of additional overhead in this algorithm. First, there is O(n) cost required to build the initial heap. The algorithm assumes the data is presented in this way. Second, extra arithmetic is required to calculate the indices of the heap subtrees. Third, while running heapify, it is possible that lg(n) elements are swapped. This incurs in additional memory overhead in each step. The elements to be swapped are not contiguous, so it might be possible that for large arrays, cache contention will play a role.

2. Implementation and comparison

The heapsort and priority queue algorithms were implemented in Python in the following repository:

https://github.com/victorvic-ucumberlands/MSCS532_Assignment4

Heapsort

To test the performance of randomized heapsort, random datasets of 100, 1000, 10000, 100000, 1000000, and 10000000 were created. Also, four arrays were created for each of the data sizes. In the first one, the data was kept random, as it was generated. In the second one, the data was sorted in ascending order (the same as the test algorithm). In the third one, the data was sorted in descending order. In the fourth one, a new randomized array with a much more limited randomization space was created for each of the data sizes. The reduced randomization space

meant the creation of arrays with many repeated elements. Figures 1 and 2 show the performance and memory consumption of heap sort, respectively. Figures 3 and 3 were obtained from a previous assignment and serve as a point of comparison between heap sort and randomized quick sort. In the first instance, it can be seen that in all the cases, randomized quicksort is faster than heapsort. This implementation of randomized quick sort handles arrays with repeated elements especially well and adds a base case for already sorted subarrays. This explains its better performance compared to heapsort. As dictated by the theory, the best, expected, and worst cases are all upper-bounded by the same O(n lg n). However, depending on the data, heapify may return earlier in some calls if it notices that the subtree already follows the heap property. For the case of sorted arrays, it is more likely that the subtrees will satisfy the heap property after each array modification, explaining its slightly better performance compared with random. For the case of the array with repeated elements, if the pool of elements downstream of a subtree is reduced, it is more likely that this subtree will comply with the heap property. While performance varies by array type, the gains are not as dramatic as with quicksort. Regarding memory consumption, there are some savings compared to quicksort, although in both cases, the increase in memory footprint because of running the algorithms is just a few KBs, which is very small for today's systems.

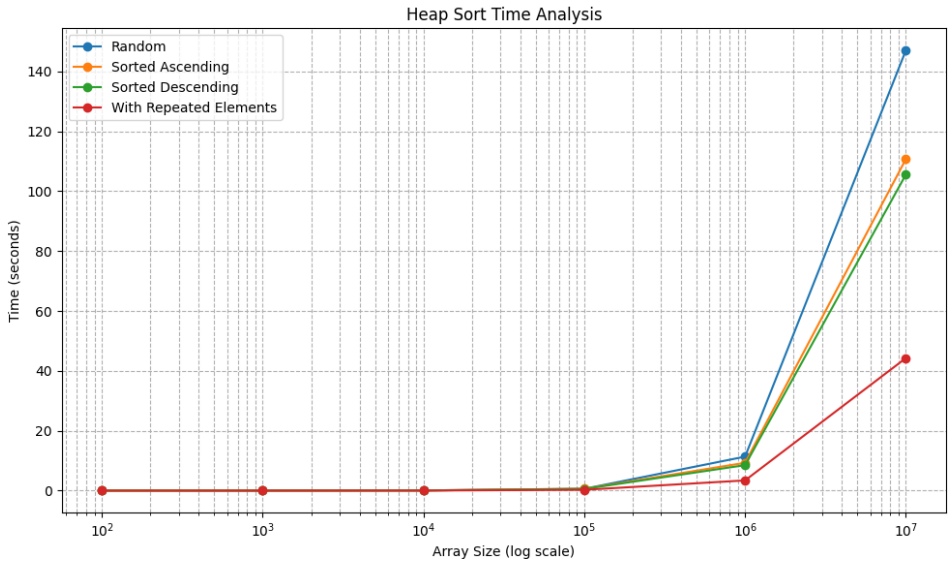

Figure 1: Performance of the heapsort algorithm with different data sizes and array types.

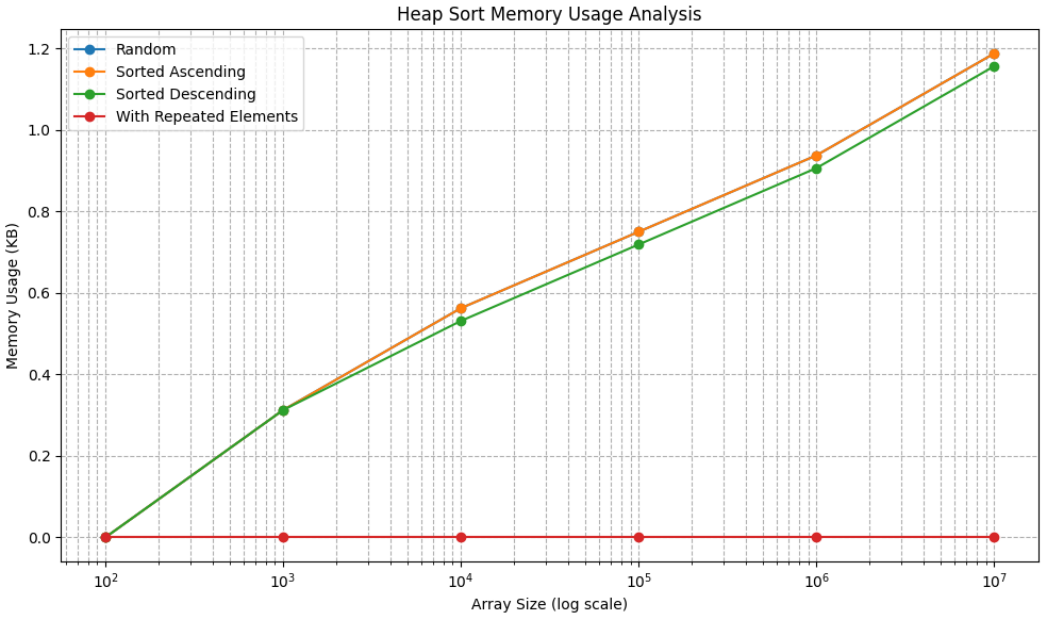

Figure 2: Memory consumption of the heapsort algorithm with different data sizes and array types

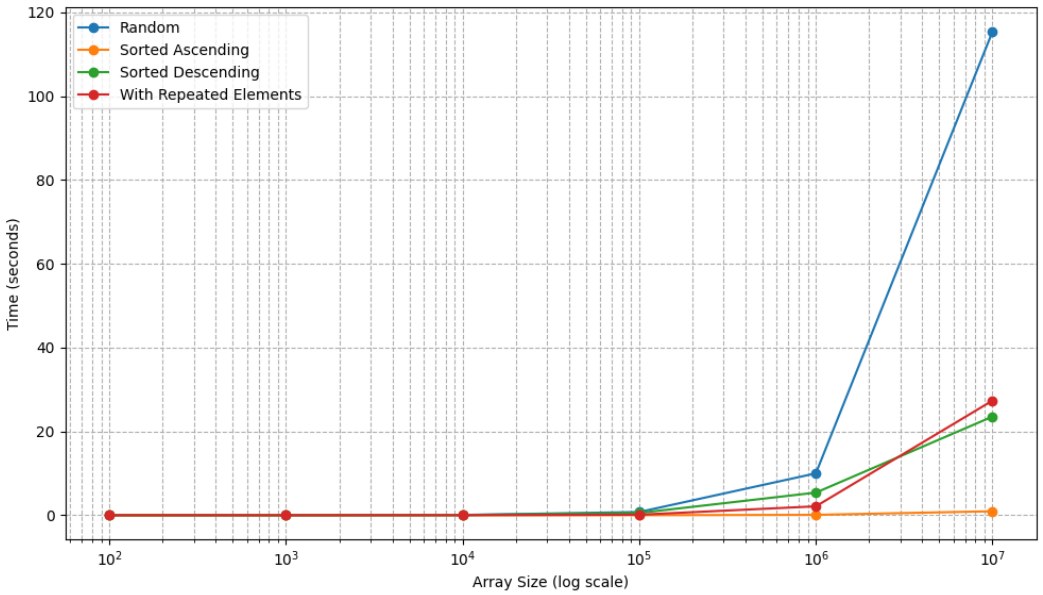

Figure 3: Performance of the randomized quick sort algorithm with different data sizes and array types.

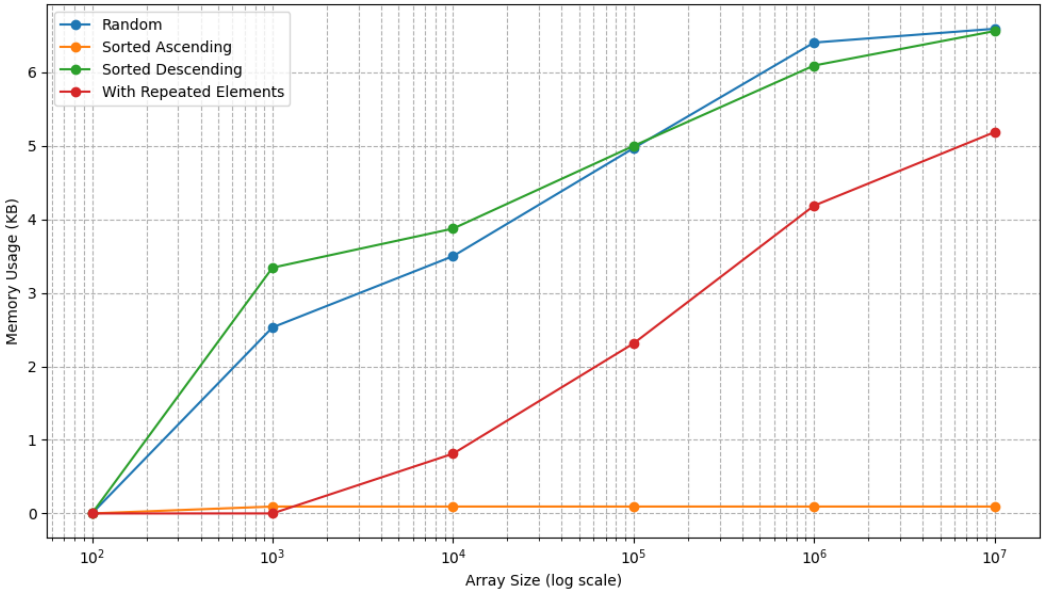

Figure 4: Memory consumption of the randomized quick sort algorithm with different data sizes and array types

Priority queue

A priority queue for a max priority scheduling algorithm was implemented in Python. Each entry of the queue contains the information <PID, priority, arrival time, deadline>. In this case, priority was selected as the variable to build the priority queue. Two data structures were created for the implementation.

Heap

The heap structure implements the priority queue. A max-heap structure was implemented as a list of pairs in Python. Each pair is <priority, PID>. The list in Python was chosen over an array because lists naturally store pointers to objects as their values. This simplifies matching each array element with its PID. If only the priority was included in the heap, then to update the hash table, it would be required to traverse the entire table to find the PID corresponding to an index in the heap. With the PID information available, this is reduced to just a regular hash value calculation.

Hashing with chaining

The hash table maps PID to the array index in the heap. It also contains other metadata, such as the arrival time and the deadline. Conflicts are resolved using chaining, and the universal hashing function implemented is $h(k) = ((a * k + b) \bmod p) \bmod m$. Hashing with chaining was chosen to store the mapping and additional metadata, since operations in the hash table are very quick, in order $O(\alpha)$. Here, α should be no greater than 1, since m is sufficiently large compared to the number of processes that could be scheduled in the system.

Insert

In the insert operation, the new element is added to the end of the array, and its final position is determined by comparing the new value with its parent and swapping if necessary. This happens at most lg (n) times. Since updating the corresponding hash table is of constant complexity, the complexity of this operation is :

$$I(n) = O(\lg(n))$$

Extract Max

In extract max, the first element of the array is obtained and then swapped with the last one. Then, the size of the array decreases by 1, and hapify is called on the root. Since hapify has complexity O(lg n) and all the other steps have constant complexity:

$$E_m = O\ (\lg(n))$$

Extract Min

In extract min, the entire heap is searched to look for the minimum value. Once it is found, this value is swapped with the last element of the heap array, the size of the array decreases by one, and heapify is called on the index of the extracted element. Since the minimum of the array is guaranteed to be in the second half of the heap, the total cost of extract is:

$$\mathrm{E}_{min} = \frac{cn}{2} + c_1 \lg(n)$$

$$\mathrm{E}_{min} = O(n)$$

Increase Key

In increase key, the index of a key is found through the hash table. Then, the key is increased, and its final position is determined by comparing the new value with its parent and swapping if necessary. As with insert, this can happen as most lg(n) times. The complexity is:

$$Inc(n) = O(\lg(n))$$

Decrease key

For decrease key, the index is found through the hash table and the priority of the element is decreased. Then, hapify is called for that specific element to restore the heap property. Since in the worst case, this can happen for the root node, the complexity is the same as hapify for the base node:

$$Dec(n) = O(\lg(n)$$

References

Cormen, T., Leiserson, C., & Rivest, R. (2022). *Introduction to Algorithms* . Cambridge: The MIT Press.